

Synapse RAG Pipeline — Full Architecture Blueprint

A Technical Reference for the Engineering Team

Document Purpose: This document consolidates every architectural decision, model choice, and system design pattern discussed during the Synapse backend design sessions. It is intended as the canonical reference for building Phase 2 of the RAG pipeline. All model names, parameter counts, and capabilities have been independently verified against official documentation and research papers as of April 2026.

Table of Contents

1. [The Big Picture](#)
 2. [The Model Stack — Verified Specs](#)
 3. [The Shared Latent Space — Nomic Text + Vision](#)
 4. [Matryoshka Representation Learning \(MRL\)](#)
 5. [Semantic Chunking — The Smart Ingestion Layer](#)
 6. [Query Classification and Semantic Routing](#)
 7. [HyDE — Hypothetical Document Embeddings](#)
 8. [The Double-Embed Strategy](#)
 9. [ColBERT, Late Interaction, and the Storage Problem](#)
 10. [Binary Quantization — The Storage Cheat Code](#)
 11. [The Ensemble Retrieval Pipeline](#)
 12. [Reciprocal Rank Fusion \(RRF\)](#)
 13. [The Cross-Encoder — Final Reranker](#)
 14. [Qdrant Configuration — Collections and Payload Design](#)
 15. [The Four Blind Spots — Phase 2 Priorities](#)
 16. [Complete System Flow Diagram](#)
 17. [Model Reference Card](#)
-

1. The Big Picture

What the team built in Phase 1 is not a basic RAG system. It is an **Ensemble Retrieval-Augmented Generation Pipeline** — a multi-model, multi-database, multi-stage retrieval system designed to eliminate the single greatest failure mode of AI search: the blind spot.

Every retrieval model has a domain where it fails. Dense vectors lose precise keywords. BM25 cannot understand synonyms. Standard ColBERT ignores broad semantic context. The architecture designed for Synapse runs all of these specialists in parallel, merges their results through a consensus algorithm, and then passes the survivors to a neural judge (the Cross-Encoder) that reads raw text to pick the absolute best chunks.

The result: a backend that cannot be stumped by how a user phrases their question.

2. The Model Stack — Verified Specs

Below is every model in the Synapse pipeline, with specs verified against official model cards and published research papers. No guesses.

`nomic-embed-text-v1.5`

- **Creator:** Nomic AI
- **Parameters:** 137 Million
- **Architecture:** Long-context BERT with Matryoshka Representation Learning
- **Context Window:** 8,192 tokens
- **Output Dimensions:** 768 (default), 512, 256, 128, 64 — all from a single model
- **Task Prefix Required:** Yes. `search_document:` for ingestion, `search_query:` for queries, `clustering:` for grouping, `classification:` for labeling.
- **License:** Apache 2.0
- **Key Fact:** Surpasses OpenAI `text-embedding-ada-002` and `text-embedding-3-small` on both short and long context retrieval tasks per MTEB benchmarks. The text model was deliberately "frozen" before the vision model was trained on top of it — this is why both models share the same embedding space without degrading text performance.

`nomic-embed-vision-v1.5`

- **Creator:** Nomic AI
- **Parameters:** 92 Million (vision encoder only)
- **Architecture:** Vision encoder trained to map into the frozen `nomic-embed-text-v1.5` embedding space
- **Output Dimensions:** 768 (default), with MRL truncation supported
- **License:** Apache 2.0
- **Key Fact:** Any image embedded with this model lands in the exact same coordinate space as text embedded with the text model. No separate image index is needed. A photo of a dark UI dashboard and the text string `"dark UI mockups"` will be mathematical neighbors in Qdrant.

`jina-colbert-v2` (aka `jinaai/jina-colbert-v2`)

- **Creator:** Jina AI
- **Parameters:** 560 Million (XLM-RoBERTa backbone with flash attention + rotary positional embeddings)
- **Architecture:** ColBERT late-interaction multi-vector model
- **Context Window:** 8,192 tokens
- **Output Dimensions (per token):** 128 (default), 96, or 64 — three separate model variants
- **Languages Supported:** 89 languages
- **Output Type:** One embedding vector per input token (multi-vector matrix)
- **License:** Apache 2.0
- **Benchmark:** Outperforms original `colbertv2.0` (Stanford) by +6.5% nDCG@10 on BEIR, outperforms `jina-colbert-v1-en` by +5.4%
- **Key Fact:** Also implements MRL, so you can reduce per-token dimensions from 128 → 64 with negligible retrieval accuracy loss and 50% storage reduction. Requires `einops` and `flash_attn`. Use via `ragatouille` or `pylate` library.
- **Important Correction from original sessions:** The parameter count is 560M, not 550M as previously discussed. The backbone is XLM-RoBERTa, not JinaBERT.

`all-MiniLM-L6-v2` (Semantic Router + Chunker)

- **Creator:** Sentence Transformers / Microsoft
- **Parameters:** ~22 Million
- **Architecture:** 6-layer MiniLM distilled from a larger model
- **Context Window:** 256 tokens
- **Output Dimensions:** 384
- **Speed:** Sub-10ms embedding on CPU
- **Use in Synapse:** (1) Semantic chunker during ingestion — identifies logical paragraph boundaries by computing cosine similarity between sentences. (2) Semantic router — embeds incoming user queries and compares against pre-embedded example phrases per agent, routing in under 10ms.

`cross-encoder/ms-marco-MiniLM-L-6-v2` (**Cross-Encoder Reranker**)

- **Creator:** Sentence Transformers
- **Parameters:** ~22 Million
- **Architecture:** 6-layer MiniLM fine-tuned on MS MARCO Passage Ranking dataset
- **Performance:** 74.30 nDCG@10 on TREC Deep Learning 2019
- **Speed:** ~1,800 documents/second on GPU; ~12ms per document batch on CPU
- **MRR@10:** 39.01 on MS MARCO Passage Reranking
- **Key Fact:** Accepts `(query, passage)` pairs and outputs a scalar relevance score. Does not produce embeddings — it reads and scores directly. Nearly identical performance to the 12-layer variant but 2x faster. This is the final judge in the pipeline: it sees raw text, not vectors.

3. The Shared Latent Space — Nomic Text + Vision

This is the architectural foundation that makes multimodal search in Synapse elegant rather than painful.

Most AI systems that handle both text and images require two entirely separate vector databases — one for text vectors, one for image vectors — and a secondary translation layer to cross-reference between them. Nomic solved this problem at the training level.

The process Nomic used:

1. Train `(nomic-embed-text-v1.5)` to full convergence. Lock it. Do not touch it again.
2. Train a brand-new vision encoder (`(nomic-embed-vision-v1.5)`) with a single objective: learn to map images into the vector space that the text model already defined.

The result is that both models operate on the same mathematical coordinate system. When you embed an image of a handwritten note using the vision model, the resulting vector lands next to text describing handwritten notes in the same Qdrant collection — no joins, no cross-references, no secondary lookups.

Practical implication for Synapse: If users upload screenshots, diagrams, or photos alongside their notes, those images can be stored in the same Qdrant collection as text. A text search query will retrieve relevant images automatically. No separate image pipeline, no separate index.

4. Matryoshka Representation Learning (MRL)

Both `nomic-embed-text-v1.5` and `jina-colbert-v2` implement MRL. Understanding this is essential for storage optimization.

What MRL is: A training technique where the model learns to pack the most important semantic information into the first few dimensions of its output vector. Subsequent dimensions add precision, but the core meaning is preserved even if you chop the vector short.

The practical effect: A 768-dimensional Nomic vector truncated to 256 dimensions retains almost identical retrieval performance while using 66% less storage and enabling faster vector math in Qdrant.

Recommended dimension settings for Synapse:

Content Type	Model	Recommended Dimensions	Rationale
General text notes	nomic-embed-text-v1.5	256	Excellent accuracy, minimal storage
Images	nomic-embed-vision-v1.5	256	Matches text index for cross-modal search
Long technical documents	nomic-embed-text-v1.5	768	Full precision warranted
ColBERT per-token vectors	jina-colbert-v2-64	64	Research confirmed: 128→64 = negligible accuracy loss, 50% storage saving

5. Semantic Chunking — The Smart Ingestion Layer

Before any vector is computed, incoming documents must be split into chunks. The naive approach — splitting every 512 tokens — is a destructive operation. It severs related ideas mid-argument and creates semantically incoherent chunks that poison retrieval.

Synapse uses **semantic chunking** via `all-MiniLM-L6-v2`.

How it works:

1. Split the document into individual sentences.
2. For each pair of adjacent sentences, compute their cosine similarity using MiniLM (384-dimensional vectors, sub-10ms per computation).
3. Identify "topic break points" — sentence boundaries where similarity drops below a defined threshold (typically 0.3-0.5 cosine distance).
4. Group sentences between topic break points into a single chunk.

Why this matters: The resulting chunks are logically complete units of thought. When a user asks "What is the row-level security policy for admin tables?", the chunk retrieved will contain the full explanation of that policy — not half the explanation cut off by an arbitrary word count.

Configuration note: `all-MiniLM-L6-v2` has a 256-token context window. For semantic chunking purposes, this is fine — you are comparing individual sentences, not long paragraphs. The model is already warm in RAM from its routing duties, so this incurs no additional memory cost.

6. Query Classification and Semantic Routing

The orchestrator's first job is deciding what to do with an incoming user query. Calling an LLM to make this routing decision is slow (500ms-1s overhead) and expensive. Synapse uses a **Semantic Router** built on `all-MiniLM-L6-v2` instead.

Setup (One-Time Pre-computation)

For each agent in the system, write 20-40 example phrases that represent what a user would say when they intend to use that agent. Embed all examples using MiniLM and store the resulting vectors in a Python dictionary in memory (not in Qdrant — these are tiny and must be instantly accessible).

Example phrase sets:

- **Quiz Agent:** ("Test me on...", "Give me flashcards for...", "Multiple choice question about...", "Quiz me on...")
- **Tutor Agent:** ("Explain how...", "I don't understand...", "Help me learn...", "What does X mean...")
- **Document Agent:** ("Summarize my notes on...", "Find my document about...", "What did I write about...")
- **General Agent:** ("What is...", "Tell me about...", "How does X work in general...")

Routing Execution (Per Query, <10ms)

1. User submits a query.
2. Embed the query using MiniLM.
3. Compute cosine similarity between the query vector and every pre-computed example vector.
4. The agent with the highest average similarity score across its examples receives the query.

This entire process — from receiving the query to determining the correct agent — completes in under 10ms on a CPU, consuming negligible memory. No LLM call, no token processing, no waiting.

Routing also controls HyDE

The semantic router is also the gate for HyDE activation (see Section 7). Vague, exploratory queries get routed through HyDE. Direct, specific queries bypass it entirely.

7. HyDE — Hypothetical Document Embeddings

HyDE (Hypothetical Document Embeddings) was introduced in the 2022 Stanford paper "*Precise Zero-Shot Dense Retrieval without Relevance Labels*". It addresses a structural mismatch in standard RAG: user queries are short questions, but documents in the vector database are long answers. These two types of text occupy slightly different neighborhoods in embedding space even when they are semantically about the same thing.

How HyDE works:

1. The user submits a vague query: ("something about policy for the database auth")
2. Instead of embedding that short query directly, the system passes it to a fast LLM with the instruction: "Write a short passage that would answer this question."
3. The LLM generates a plausible hypothetical answer. This answer does not need to be factually correct — it just needs to be structurally similar to the kind of content that exists in the database.
4. That hypothetical answer is embedded. Because it is a full, answer-shaped paragraph, it lands much closer to actual answer documents in vector space than the short question would have.
5. The hypothetical embedding is used for the Qdrant query instead of the original query embedding.

When to trigger HyDE in Synapse: Not always. HyDE adds latency (the LLM generation step). The semantic router should trigger HyDE only for queries that are vague, exploratory, or phrased as open-ended questions. Direct factual queries ("What is the command to start Uvicorn?") skip HyDE entirely and go straight to vector search.

Known HyDE trade-off (verified by 2025 research): If the LLM is small or poorly aligned, the hypothetical document can hallucinate enough to send the search in the wrong direction entirely. A hybrid fallback policy is recommended: run HyDE, but also run the standard query embedding in parallel. If HyDE retrieval similarity score is below a threshold, fall back to the standard query result. This "adaptive HyDE" pattern has been shown to outperform pure HyDE.

8. The Double-Embed Strategy

This is the core architectural innovation of the Synapse pipeline. Instead of choosing between dense vectors (Nomic) and multi-vectors (Jina ColBERT), complex and technically dense content is routed through both at ingestion time.

Ingestion Routing Decision

When a document arrives, the orchestrator classifies it before embedding:

Low-Complexity Path (Dense Only):

- General notes, flashcard content, simple summaries
- Embedded via (nomic-embed-text-v1.5) only
- One vector stored in (Collection_Nomic) in Qdrant
- Cost: cheap, fast, minimal storage

High-Complexity Path (Double-Embed):

- Technical PDFs, code documentation, system design notes, reference manuals
- Embedded via `nomic-embed-text-v1.5` → stored in `Collection_Nomic`
- Also embedded via `jina-colbert-v2-64` → stored in `Collection_ColBERT`
- Cost: slightly more CPU time and storage, but both models run against already-warm model instances in RAM (Singleton pattern)

Deduplication via Payload Key

The critical linking mechanism: every point stored in Qdrant — in either collection — carries the same `postgres_doc_id` in its payload. This UUID references the canonical document row in PostgreSQL. When parallel search results come back from both collections, the orchestrator uses this field to deduplicate: if Nomic and Jina both found the same chunk, it appears once in the merged result set, not twice.

Why This Architecture Covers Its Own Blind Spots

Scenario	Nomic Result	Jina ColBERT Result
Vague conceptual query	Finds relevant content	May miss if phrasing differs
Exact variable name / command search	May dilute the term into surrounding context	Finds it precisely via MaxSim
Mixed query (concept + specific keyword)	Handles concept part	Handles keyword part
Both models find same doc	Deduplicated	Confidence boosted in RRF

9. ColBERT, Late Interaction, and the Storage Problem

Why ColBERT Exists

Standard dense embedding models (like Nomic Text) read an entire paragraph and "squash" all its meaning into a single point in vector space. This is computationally elegant but lossy. A highly specific variable name like `POSTGRES_MAX_CONNECTIONS` gets diluted by surrounding words. The vector for a paragraph containing that variable drifts toward the general topic of PostgreSQL configuration rather than pointing precisely at that exact term.

ColBERT uses a different architecture: **Late Interaction** via the **MaxSim** operator.

ColBERT at ingestion: Instead of one vector per paragraph, it produces one vector per token. A 200-token paragraph becomes a 200-vector matrix stored in Qdrant.

ColBERT at query time: The query is also decomposed into per-token vectors. For each query token, the system scans all document tokens and finds the one with the highest similarity (MaxSim). The final relevance score is the sum of all these per-token maximum matches.

The result: ColBERT cannot be confused by surrounding context. It finds

`POSTGRES_MAX_CONNECTIONS` in a document even if that document also talks about 50 other unrelated PostgreSQL concepts.

The Storage Problem — Real Numbers

For a 200-token paragraph:

- **Nomic Dense (768D, float32):** 200 tokens $\rightarrow$ 1 vector $\rightarrow 768 \times 4$ bytes $\approx$ **3 KB**
- **Jina ColBERT (128D per token, float32):** 200 tokens $\rightarrow$ 200 vectors $\rightarrow 200 \times 128 \times 4$ bytes $\approx$ **102 KB**
- **Jina ColBERT-64 (64D per token, float32):** 200 tokens $\rightarrow$ 200 vectors $\rightarrow 200 \times 64 \times 4$ bytes $\approx$ **51 KB**
- **Jina ColBERT-64 + Binary Quantization:** $200 \times 64 \times 1$ bit $\approx$ **1.6 KB** — comparable to dense!

This is why binary quantization is not optional for a ColBERT deployment — it is the mechanism that makes the architecture viable at all (see Section 10).

Zero-Shot Generalization — ColBERT's Hidden Superpower

Dense models that are trained on one domain (say, medical literature) struggle when deployed on a different domain (say, Arch Linux system commands) because their embedding space was shaped by the training distribution. ColBERT models — because they operate at the token level rather than the semantic-cluster level — perform well on domains they have never seen before. This is why Jina ColBERT v2's BEIR benchmark numbers hold up across diverse dataset types including code, Q&A, fact retrieval, and technical documentation.

For Synapse specifically: users store notes across wildly different subjects. A dense model trained primarily on English web text will lose precision on niche technical jargon. Jina ColBERT will not.

10. Binary Quantization — The Storage Cheat Code

Binary quantization is the engineering technique that makes running a ColBERT index on local hardware feasible.

How It Works

Each dimension of a standard vector is stored as a 32-bit float (`float32`). Binary quantization discards the magnitude of each value and keeps only its sign:

- Positive value → **1**
- Negative value → **0**

A 128-dimensional ColBERT vector shrinks from 512 bytes (128×4 bytes) to 16 bytes (128 bits), a **32x compression factor**.

Why It Doesn't Destroy Accuracy

Models like `jina-colbert-v2` are specifically trained with this compression in mind. The training process encourages the model to push meaningful signal into the sign of each dimension rather than distributing it evenly across the magnitude. The binary approximation of the MaxSim operation (now computed via **Hamming distance** — counting differing bits rather than computing cosine similarity) correlates closely with the full-precision score. In practice, retrieval accuracy degrades by only 2-5% while storage drops by 32x.

Qdrant's Native Implementation

As of Qdrant version 1.10 (released July 2024), native multivector (ColBERT) support is built into the core engine. No custom pre-processing is required.

Qdrant handles binary quantization in two layers:

1. **RAM layer:** Stores the binarized (1-bit) vectors. Used for fast candidate retrieval via Hamming distance during HNSW graph traversal.
2. **Disk layer:** Stores the original float32 vectors. Used for rescoring top candidates after initial retrieval (the "rescore" step recovers accuracy lost in binarization).

This asymmetric architecture — coarse binary search + precise float32 rescoring — gives near-full-precision results at storage costs close to a binary-only system.

Qdrant collection configuration for ColBERT:

python

```
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance, MultiVectorConfig,
    MultiVectorComparator, BinaryQuantization, BinaryQuantizationConfig
)

client.create_collection(
    collection_name="synapse_colbert",
    vectors_config=VectorParams(
        size=64, # jina-colbert-v2-64 dimensions
        distance=Distance.COSINE,
        multivector_config=MultiVectorConfig(
            comparator=MultiVectorComparator.MAX_SIM
        ),
        quantization_config=BinaryQuantization(
            binary=BinaryQuantizationConfig(always_ram=True)
        )
    )
)
```

11. The Ensemble Retrieval Pipeline

The ensemble pipeline is the orchestration layer that runs all three retrieval strategies in parallel and merges the results.

The Three Specialists

Specialist A — BM25 Sparse Search (Qdrant Sparse Vectors)

- What it finds: Documents containing the exact words from the query
- Strength: Perfect recall for precise terminology, variable names, error codes
- Weakness: Cannot understand synonyms or paraphrases. "canine" misses "dog"
- Qdrant implementation: Native sparse vector support (available since Qdrant 1.7)

Specialist B — Nomic Dense Search

- What it finds: Documents semantically similar to the query, regardless of exact wording
- Strength: Handles natural language, paraphrases, conceptual queries
- Weakness: Dilutes highly specific technical terms into surrounding semantic context
- Qdrant implementation: Standard dense vector collection with cosine similarity

Specialist C — Jina ColBERT Multi-Vector Search

- What it finds: Documents with the highest per-token semantic overlap with query tokens
- Strength: Zero-shot generalization, resistant to domain shift, precise on technical jargon
- Weakness: Storage intensive; slower than dense search due to MaxSim computation
- Qdrant implementation: Multivector collection with MaxSim comparator + binary quantization

Parallel Execution

All three searches are fired simultaneously using `asyncio.gather()` in the FastAPI backend. The total wall-clock time of the retrieval stage equals the slowest of the three queries — not their sum.

python

```
nomic_results, bm25_results, colbert_results = await asyncio.gather(
    search_nomic_collection(query_embedding, top_k=10),
    search_bm25_collection(sparse_query_vector, top_k=10),
    search_colbert_collection(query_multivector, top_k=10)
)
```

Each search returns the top 10 results. The pipeline now holds up to 30 candidate chunks (with potential duplicates across collections, handled by RRF).

12. Reciprocal Rank Fusion (RRF)

After parallel retrieval, there is a mathematical incompatibility problem: the three search systems use completely different scoring scales.

- BM25 scores: arbitrary TF-IDF-weighted floats (e.g., `14.8`)
- Nomic cosine similarity: values between -1 and 1 (e.g., `0.87`)
- ColBERT MaxSim: sum of per-token maximum cosine similarities (e.g., `112.4`)

These numbers cannot be compared or averaged directly. RRF solves this by ignoring the raw scores entirely and working only with rank positions.

The RRF formula for a document `d`:

```
RRF_score(d) = Σ 1 / (k + rank_i(d))
```

Where `k` is a smoothing constant (typically 60), and `rank_i(d)` is the position of document `d` in retrieval system `i`'s result list (1 = first place).

Example: Document X is ranked #1 by Nomic, #4 by BM25, and #2 by ColBERT:

$$\text{RRF}(X) = 1/(60+1) + 1/(60+4) + 1/(60+2) = 0.01639 + 0.01563 + 0.01613 = 0.04815$$

Document Y is ranked #1 by BM25 but not found by the other two:

$$\text{RRF}(Y) = 0 + 1/(60+1) + 0 = 0.01639$$

Document X wins the consensus despite not being #1 in any single system. This is the core value of RRF: it rewards consistent relevance across multiple retrieval methods over dominance in just one.

After RRF scoring, the 30 raw results are deduplicated (using `postgres_doc_id`) and sorted by RRF score. The top 10 unique chunks proceed to the Cross-Encoder.

13. The Cross-Encoder — Final Reranker

The final stage of retrieval before generation is the Cross-Encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`).

Why a Cross-Encoder After RRF

RRF is a statistical consensus algorithm. It is mathematically elegant but it does not read text — it just counts ranks. Two documents could have identical RRF scores while one is perfectly relevant and one is topically adjacent but useless. The Cross-Encoder closes this gap.

How It Works

A Cross-Encoder is fundamentally different from a bi-encoder (like Nomic or MiniLM for embeddings). It does not encode query and document independently and compare vectors. Instead, it concatenates the query and document into a single input:

```
[CLS] user query [SEP] candidate document text [SEP]
```

The model processes this combined input with full self-attention — every word in the query attends to every word in the document simultaneously. The output is a single scalar relevance score. This is the same mechanism used by BERT-based models for reading comprehension, adapted for ranking.

Practical consequence: The Cross-Encoder cannot be fooled by keyword overlap or vector similarity coincidences. It understands causality, negation, conditional logic, and nuanced relevance that a bi-encoder will miss.

The final step: The Cross-Encoder scores all 10 RRF-filtered candidates and returns the top 3. These 3 chunks are what gets included in the LLM context for answer generation.

Performance Note

At ~1,800 documents/second on GPU and ~12ms per 10-document batch on CPU, the Cross-Encoder adds minimal latency for the small candidate sets Synapse produces (10 chunks → typically under 50ms on CPU).

14. Qdrant Configuration — Collections and Payload Design

Recommended Collection Structure

`synapse_dense` (Nomic Text + Nomic Vision)

```
python

VectorParams(
    size=256,          # MRL truncated – excellent accuracy, low storage
    distance=Distance.COSINE
)
```

`synapse_sparse` (BM25)

```
python

SparseVectorParams(
    index=SparseIndexParams(on_disk=False)
)
```

`synapse_colbert` (Jina ColBERT-64 + Binary Quantization)

```
python

VectorParams(
    size=64,
    distance=Distance.COSINE,
    multivector_config=MultiVectorConfig(comparator=MultiVectorComparator.MAX_SIM),
    quantization_config=BinaryQuantization(binary=BinaryQuantizationConfig(always_ra
)
```

Payload Schema (All Collections)

Every Qdrant point — regardless of which collection — must carry this payload:

```
json
{
  "postgres_doc_id": "uuid-v4-from-postgres",
  "user_id": "uuid-of-document-owner",
  "chunk_index": 3,
  "content_type": "text | image | code",
  "complexity_tier": "standard | high",
  "source_title": "document title",
  "created_at": "2025-01-01T00:00:00Z"
}
```

`postgres_doc_id`: Used for deduplication in RRF and for fetching the full document from PostgreSQL when building the LLM context.

`user_id`: Used for **hard filtering in Qdrant before the vector search runs**. This is non-negotiable for a multi-user system. Configure a Qdrant payload index on `user_id` (the index converts this from a linear scan to a hash lookup):

```
python
client.create_payload_index(
    collection_name="synapse_dense",
    field_name="user_id",
    field_schema=PayloadSchemaType.KEYWORD
)
```

Every search call must include a filter:

```
python
query_filter=Filter(
    must=[FieldCondition(key="user_id", match=MatchValue(value=current_user_id))]
)
```

This filter is applied during HNSW graph traversal in Qdrant — it does not degrade search quality because Qdrant's one-stage filtered search was explicitly engineered for this pattern.

15. The Four Blind Spots — Phase 2 Priorities

These are the four areas that have not been fully architected yet. They represent the gap between a retrieval engine and a production application.

Blind Spot 1: Context Window Management (LLM Payload Construction)

After the Cross-Encoder returns the top 3 chunks, these chunks must be assembled into the LLM's context window correctly. Issues to resolve:

- **Token budget:** The top 3 chunks from Jina ColBERT could be up to 8,192 tokens each if full-length documents are indexed. A max token budget must be enforced before LLM submission (recommend: 3,000-4,000 tokens for retrieved context, reserving the rest for system prompt + user message + response).
- **Chunk ordering:** Research shows LLMs perform better when the most relevant chunk is placed last in the context window ("lost in the middle" problem — models tend to underweight content in the middle of long contexts). Sort chunks so the highest Cross-Encoder score chunk appears closest to the query.
- **System prompt templating:** The prompt wrapper must clearly delineate retrieved context from the user query. A structured template like `---CONTEXT---\n{chunks}\n---\nQUESTION---\n{query}` consistently outperforms unformatted injection.

Blind Spot 2: Async Ingestion Queue

Running a large PDF through semantic chunking → Nomic embedding → Jina ColBERT embedding is a multi-second CPU operation. If done synchronously in the FastAPI request handler, it blocks the event loop and makes the upload endpoint unresponsive.

Recommended solution: Use FastAPI's `BackgroundTasks` for lightweight documents and a proper task queue (Celery with Redis, or ARQ) for heavy ones. The endpoint returns `{"status": "processing", "doc_id": "..."}` immediately. The frontend polls a `/status/{doc_id}` endpoint or subscribes to a WebSocket channel that fires when ingestion completes.

Blind Spot 3: Hard Multi-Tenant Filtering

As noted in Section 14, user isolation must be enforced at the Qdrant query level using payload filters — not at the application layer after retrieval. The Qdrant payload index on `user_id` (a KEYWORD type index) reduces this to a near-zero-cost operation that runs during graph traversal. Without this, a vector similarity search could theoretically surface another user's documents if their content is semantically similar.

Blind Spot 4: Feedback Loop Architecture

The `feedback_loops_enabled` flag in the Synapse startup logs suggests this was planned but not implemented. Here is the recommended design:

Signal collection: Log the following events to a `retrieval_feedback` table in PostgreSQL:

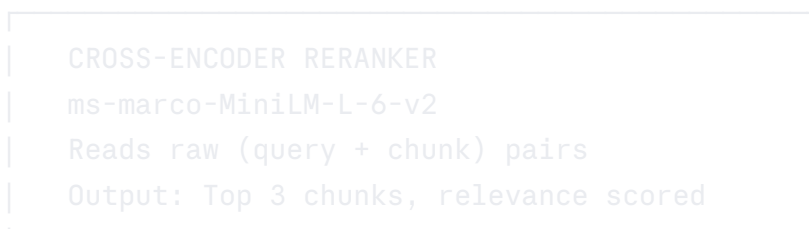
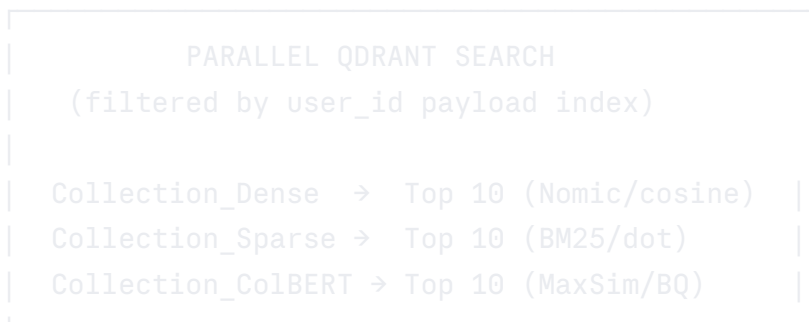
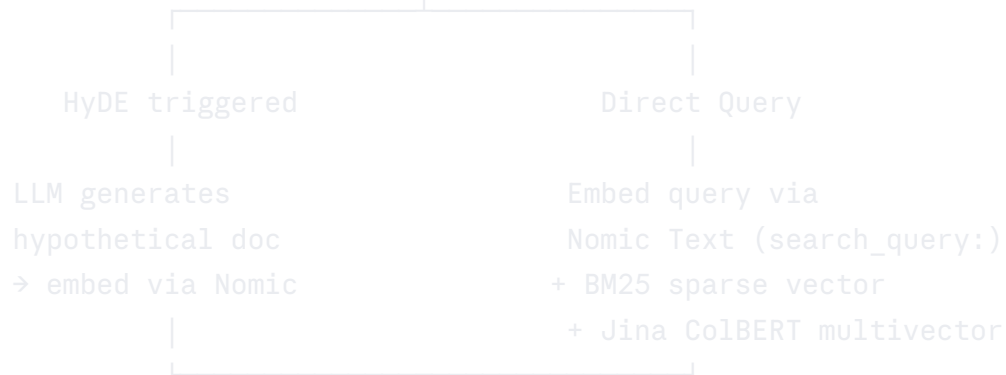
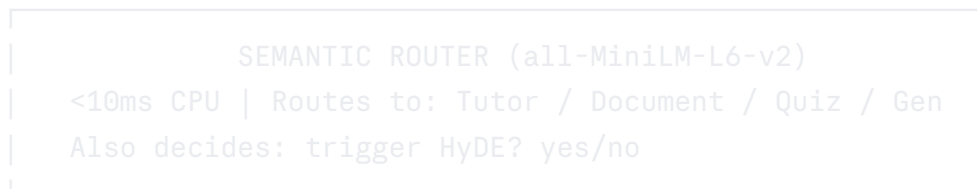
- `query_id`, `returned_doc_ids` (list), `user_accepted_doc_id` (which chunk the LLM used), `user_rated_response` (thumbs up/down), `timestamp`

Short-term use: Use thumbs-down events to flag specific chunks for re-embedding or removal from the Qdrant index.

Long-term use: Accumulate positive examples (query + accepted chunk pairs) for fine-tuning the Cross-Encoder on Synapse-specific content. Even 500-1,000 labeled pairs can measurably improve reranking precision on a domain-specific corpus.

16. Complete System Flow Diagram

USER QUERY



CONTEXT ASSEMBLY

Token budget enforcement (<4,000 tokens)

Chunk ordering (best chunk last)

System prompt templating



LLM GENERATION

(via LiteLLM router)



RESPONSE TO USER

+ log to feedback table

17. Model Reference Card

A quick-reference table for the entire Synapse model stack.

Model	Role	Parameters	Context	Output Size	RAM Footprint	CPU Viable
<code>nomic-embed-text-v1.5</code>	Dense embedder (text)	137M	8,192 tok	256–768D	~550MB	✓ Yes
<code>nomic-embed-vision-v1.5</code>	Dense embedder (images)	92M	N/A (images)	256–768D	~370MB	✓ Yes
<code>jina-colbert-v2-64</code>	Multi-vector embedder	560M	8,192 tok	64D/token	~1.1GB	✓ Yes
<code>all-MiniLM-L6-v2</code>	Router + chunker	~22M	256 tok	384D	~90MB	✓ Fast
<code>ms-marco-MiniLM-L-6-v2</code>	Cross-encoder reranker	~22M	512 tok	scalar score	~90MB	✓ Fast

Total RAM for full model stack (all warm): approximately 2.2GB — manageable on any server with 4GB+ RAM.

Appendix A: Library and Dependency Reference

```
# Core embedding and reranking
sentence-transformers>=2.7.0
torch>=2.2.0

# Jina ColBERT
einops>=0.7.0
flash-attn>=2.5.0
ragatouille>=0.0.8      # OR pylate>=1.1.0

# Vector database
qdrant-client>=1.10.0  # 1.10+ required for native multivector support

# API and serving
fastapi>=0.111.0
uvicorn[standard]>=0.29.0

# Task queue (for async ingestion)
celery>=5.4.0  # OR arq>=0.25.0
redis>=5.0.0

# LLM routing
litellm>=1.40.0
```

Appendix B: Key Architecture Decisions Summary

Decision	Choice	Reason
Text embedding	nomic-embed-text-v1.5 at 256D	MRL lets us halve storage with negligible accuracy loss
Multi-vector embedding	jina-colbert-v2-64	560M params but 64D per token keeps storage viable; 8192 context handles long docs
Quantization	Binary (1-bit) on ColBERT	32x compression; Hamming distance search is microsecond-speed on CPU
Routing	Semantic (MiniLM cosine similarity)	<10ms vs 500-1000ms LLM-based routing
Result merging	RRF (k=60)	Score-agnostic rank fusion; standard in production hybrid search
Final ranking	Cross-Encoder	Only model that reads the text rather than comparing vector geometry
User isolation	Qdrant payload filter on user_id	Filter applied during HNSW traversal; zero accuracy degradation
Ingestion chunking	Semantic (cosine sentence similarity)	Preserves logical unit integrity; avoids severed arguments

Document compiled from design sessions and independently verified against official model documentation, Hugging Face model cards, Jina AI research paper (arXiv:2408.16672), Nomic AI documentation, Qdrant v1.10 release notes, and Sentence Transformers official benchmarks. Last updated: April 2026.